Olist E-commerce Lakehouse

End-to-End Data Engineering Project on Databricks

Author: Gnana Sushmitha Tammisetti

Email: gnanasushmitha@gmail.com

LinkedIn: https://www.linkedin.com/in/gnanasushmitha

GitHub: https://github.com/sushii-code/azure-lakehouse-sales

Project Type: Portfolio / Data Engineering Project

Tools Used: Databricks (Unity Catalog, Delta Lake, Jobs, AI/BI), PySpark, dbt Core, Terraform, GitHub Actions, Python

Year: 2026

Contents

1. Project Overview
2. Problem Statement
3. Technology Stack
4. Solution Architecture
5. Data Characteristics
6. Unity Catalog Setup
7. Bronze Layer: Raw Ingestion
8. Silver Layer: Cleaning, Quality Rules and Quarantine
9. GDPR: Personal Data Tokenisation
10. dbt Project Setup and Structure
11. Gold Layer: dbt Star Schema
12. Testing and Reconciliation
13. Workflow Orchestration with Databricks Jobs
14. CI/CD with GitHub Actions
15. Infrastructure as Code with Terraform
16. Dashboard with Databricks AI/BI
17. Key Learnings and Challenges
18. Conclusion
19. Future Enhancements

1. Project Overview

The goal of this project is to design and build an end-to-end lakehouse that turns raw e-commerce files into trusted, analytics-ready data, and to prove at every step that no data is lost, duplicated or silently changed.

The project uses the public Olist Brazilian e-commerce dataset (99,441 orders from 2016 to 2018) and runs on Databricks Free Edition. It integrates:

- Layered medallion modelling (Bronze, Silver, Gold) on Delta Lake and Unity Catalog
- Data quality rules with quarantine tables and a full bronze-to-gold reconciliation report
- A dbt star schema with an SCD Type 2 customer dimension and an incremental fact table
- GDPR-style handling of personal data through a restricted schema and a token vault
- Automated orchestration with a 6-task Databricks Job
- CI/CD with GitHub Actions and Infrastructure as Code with Terraform
- A business dashboard built with Databricks AI/BI

Headline results:

Result	Value
Delivered revenue on the dashboard	13,220,248.93 BRL
Source rows accounted for, bronze to gold	100% (clean, quarantined, deduplicated or deliberately excluded)
Automated dbt tests	44 (latest build: PASS=60, WARN=1, ERROR=0)
End-to-end pipeline run	4 min 41 s as a 6-task Databricks Job
Incremental load proof	6,566 deleted fact rows restored exactly (back to 112,635)
Infrastructure as Code	6 schemas and 1 volume imported into Terraform; plan shows No changes

2. Problem Statement

In real e-commerce companies, sales data arrives from several systems and is:

- Messy: dates and money arrive as text, with typos, duplicates and invalid values
- Spread across many related tables (orders, items, payments, reviews, customers, products, sellers)
- Full of personal information that must be protected under UK GDPR
- Used by analysts and managers who need numbers they can trust

Simple load-and-report pipelines fall short because they:

- Drop bad rows silently, so totals no longer match the source
- Join tables at the wrong grain, which inflates revenue (fan-out)

- Lose history when a customer's details change
- Expose personal data to everyone who can query the warehouse
- Depend on manual steps that are hard to repeat and easy to break

This project therefore addresses the question:

How can raw e-commerce files be turned into a trusted, private, automated and fully reconciled star schema, where every number on the dashboard can be traced back to the source?

3. Technology Stack

Platform and Storage

- Databricks Free Edition (serverless compute)
- Unity Catalog: catalog, schemas, volumes and governance
- Delta Lake: ACID tables with MERGE support

Processing

- PySpark: bronze ingestion, silver cleaning and PII tokenisation
- Spark SQL: setup scripts and analysis

Modelling and Testing

- dbt Core 1.12 with the dbt-databricks adapter
- dbt_utils: surrogate keys
- Built-in and custom (singular) dbt tests

Orchestration and DevOps

- Databricks Jobs (Lakeflow): task graph, schedule and retries
- GitHub Actions: linting, dbt parse, secret scanning and Terraform checks
- Terraform: Databricks provider (deployed) and azurerm provider (validated)

Data Privacy

- Faker (seeded) for synthetic personal data
- UUID token vault in a restricted schema

Visualisation

- Databricks AI/BI dashboard on the gold layer

This toolset reflects a modern lakehouse stack used by data engineering teams on Databricks and Azure.

4. Solution Architecture

Data moves through four layers in one Unity Catalog catalog, `workspace`. PySpark builds bronze and silver, and dbt builds staging, gold and the audit report. Personal data is kept apart in a restricted

schema, and only tokens flow into the analytics layers.

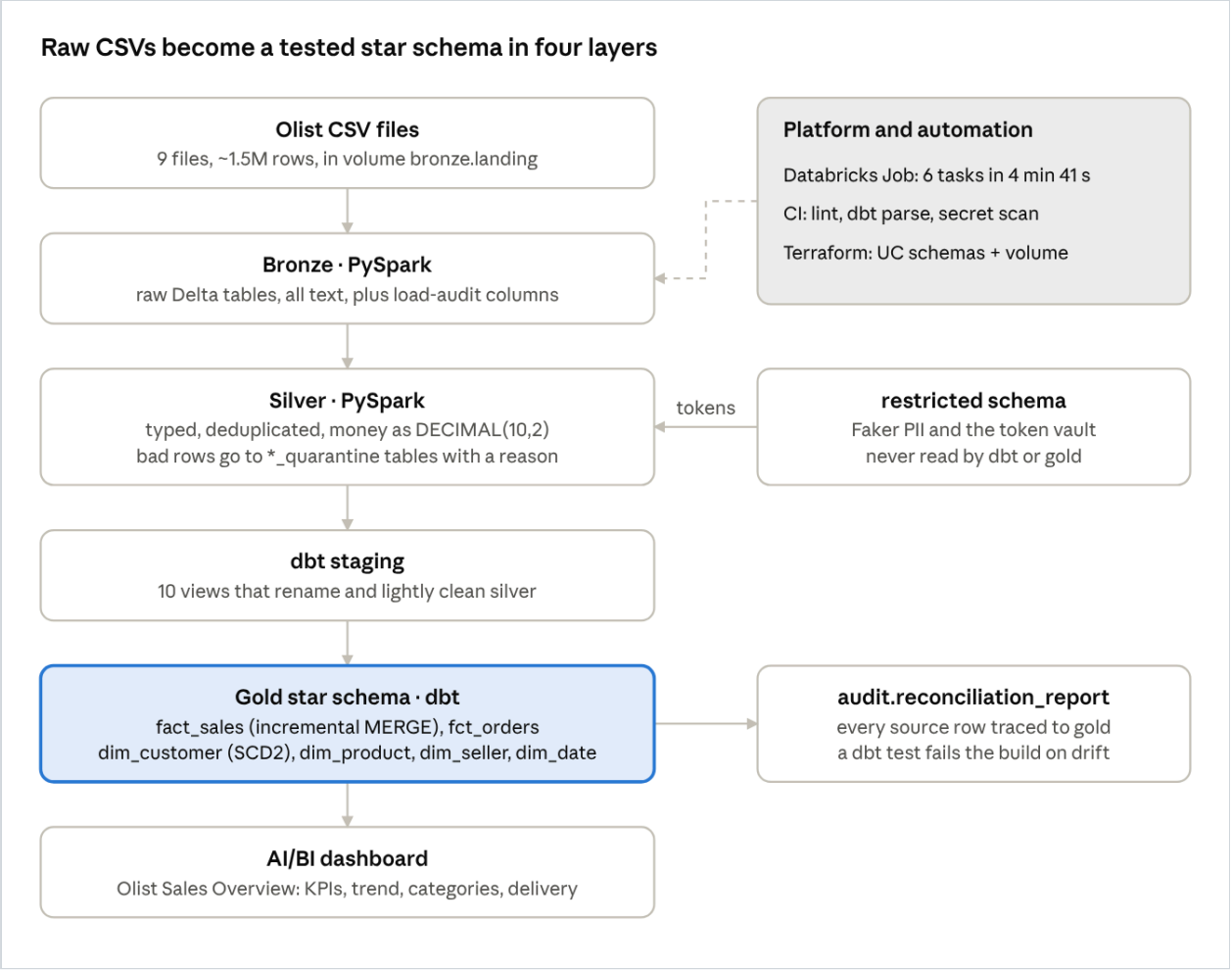


Figure 1. End-to-end architecture

The platform tools act on every layer rather than one step: the Databricks Job runs the pipeline, GitHub Actions checks every change, and Terraform defines the catalog structure.

Layer Responsibilities

Layer	Built by	Purpose
Bronze	PySpark	Raw data exactly as received, plus load-audit columns
Silver	PySpark	Typed, deduplicated, quality-checked data; bad rows quarantined
Restricted	PySpark	Synthetic personal data and the token vault
Staging	dbt (views)	Renamed and lightly cleaned silver sources
Gold	dbt (tables)	Business-ready star schema
Audit	dbt (table)	Reconciliation report from bronze to gold

5. Data Characteristics

5.1 Source of Data

The data comes from the Olist Brazilian e-commerce public dataset, which describes orders placed on a marketplace connecting small sellers to customers. It contains 9 CSV files that together form a normalised, relational model:

- **Orders**: one row per order with its status and 5 lifecycle timestamps
- **Order items**: one row per item, with product, seller, price and freight
- **Payments**: one or more payments per order (card, boleto, voucher, debit card)
- **Reviews**: customer review scores and comments
- **Customers, Sellers, Products**: descriptive (dimension) data
- **Geolocation**: latitude and longitude for Brazilian postcode prefixes
- **Category translation**: Portuguese to English product category names

5.2 Data Volumes

Table	Rows	Grain
geolocation	1,000,163	One map point (many per postcode prefix)
order_items	112,650	One item in an order
payments	103,886	One payment for an order
orders	99,441	One order
customers	99,441	One customer ID per order
reviews	99,224	One review
products	32,951	One product
sellers	3,095	One seller
category_translation	71	One category

5.3 Key Table Schemas

The tables below show the main columns after typing in silver.

Orders Table

Column	Data Type	Description
order_id	STRING	Unique order identifier
customer_id	STRING	Customer ID for this order
order_status	STRING	delivered, shipped, canceled, invoiced and others
order_purchase_timestamp	TIMESTAMP	When the order was placed
order_approved_at	TIMESTAMP	When payment was approved
order_delivered_carrier_date	TIMESTAMP	Handed to the carrier
order_delivered_customer_date	TIMESTAMP	Delivered to the customer
order_estimated_delivery_date	TIMESTAMP	Promised delivery date

Order Items Table

Column	Data Type	Description
order_id	STRING	Foreign key to orders
order_item_id	INT	Item number within the order
product_id	STRING	Foreign key to products
seller_id	STRING	Foreign key to sellers
shipping_limit_date	TIMESTAMP	Seller's shipping deadline
price	DECIMAL(10,2)	Item price in BRL
freight_value	DECIMAL(10,2)	Shipping cost in BRL

Payments Table

Column	Data Type	Description
order_id	STRING	Foreign key to orders
payment_sequential	INT	1st, 2nd, 3rd payment for the order
payment_type	STRING	credit_card, boleto, voucher, debit_card
payment_installments	INT	Number of instalments
payment_value	DECIMAL(10,2)	Amount paid in BRL

Customers Table

Column	Data Type	Description
`customer_id`	`STRING`	ID created for each order
`customer_unique_id`	`STRING`	The real person (stable across orders)
`customer_zip_code_prefix`	`STRING`	5-character postcode prefix (leading zeros kept)
`customer_city`	`STRING`	City, trimmed and lower-case
`customer_state`	`STRING`	Two-letter state code

These schemas ensure:

- Financial precision using `DECIMAL(10,2)` instead of floating point
- Postcodes kept as text, so leading zeros are not lost
- Lifecycle tracking through typed timestamps
- Referential integrity between orders, items, products, sellers and customers, tested by dbt

6. Unity Catalog Setup

Before ingestion, the lakehouse structure was created inside the `workspace` catalog. Every statement uses `IF NOT EXISTS`, so the setup is safe to rerun (idempotent).

00_setup.py (Cell 1)

```
-- A schema is a folder for tables inside the "workspace" catalog.
CREATE SCHEMA IF NOT EXISTS workspace.bronze;   -- raw data, exactly as received
CREATE SCHEMA IF NOT EXISTS workspace.silver;   -- cleaned, typed, deduplicated data
CREATE SCHEMA IF NOT EXISTS workspace.gold;     -- business-ready tables for reporting

-- A volume stores FILES (like CSVs). "landing" is where raw files first arrive.
CREATE VOLUME IF NOT EXISTS workspace.bronze.landing;
```

The `restricted` schema is created by the PII notebook, and dbt creates `staging` and `audit` on its first run. All 6 schemas and the volume were later imported into Terraform (Section 15).

Schema	Role
bronze	Raw tables, the landing volume and simulated address changes
silver	Clean tables, quarantine tables and tokenised customer profiles
restricted	Synthetic personal data and the token vault
staging	dbt staging views
gold	Star schema: 2 fact tables and 4 dimensions
audit	Reconciliation report

7. Bronze Layer: Raw Ingestion

Bronze keeps the data exactly as received. Every column stays as text, and two audit columns record when and from which file each row was loaded. One loop loads all 9 files.

01_bronze_ingest.py (Cell 3)

```
LANDING = "/Volumes/workspace/bronze/landing/olist"

tables = {
    "orders": "olist_orders_dataset.csv",
    "order_items": "olist_order_items_dataset.csv",
    "customers": "olist_customers_dataset.csv",
    "payments": "olist_order_payments_dataset.csv",
    "reviews": "olist_order_reviews_dataset.csv",
    "products": "olist_products_dataset.csv",
    "sellers": "olist_sellers_dataset.csv",
    "geolocation": "olist_geolocation_dataset.csv",
    "category_translation": "product_category_name_translation.csv",
}

for table, file in tables.items():
    df = (spark.read
          .option("header", True)          # first line = column names
          .option("multiLine", True)       # review comments can contain line breaks
          .option("escape", '"')           # "" inside quoted text is a literal quote
          .csv(f"{LANDING}/{file}")
          .withColumn("_ingested_at", F.current_timestamp())          # WHEN it was
loaded
          .withColumn("_source_file", F.col("_metadata.file_path")))   # WHICH file it
came from

    (df.write
       .mode("overwrite")                     # safe to rerun (idempotent)
       .option("overwriteSchema", True)
       .saveAsTable(f"workspace.bronze.{table}"))
```

Why these options matter: without `multiLine` and `escape`, review comments that contain line breaks or quotes would be split across several rows, and the reviews table would no longer match the source file.

Bronze Reconciliation

A final cell compares each bronze table with the row counts measured on the source files. All 9 tables matched.

01_bronze_ingest.py (Cell 4)

```
for table, exp in expected.items():
    actual = spark.table(f"workspace.bronze.{table}").count()
    status = "OK" if actual == exp else "MISMATCH"
    print(f"{table:22} expected {exp:>9,}   bronze {actual:>9,}   {status}")
```

8. Silver Layer: Cleaning, Quality Rules and Quarantine

Silver converts text into real types, applies quality rules and sets bad rows aside with a reason instead of deleting them. This means every bronze row ends up in exactly one place: the clean table or its quarantine table.

8.1 Typing Without Crashing

`try_to_timestamp` and `try_cast` return NULL instead of failing the whole job when a value cannot be converted. Because a failure is silent, the notebook then counts failed conversions explicitly.

build_silver (Cells 1-2)

```
order_ts_cols = [
    "order_purchase_timestamp", "order_approved_at",
    "order_delivered_carrier_date", "order_delivered_customer_date",
    "order_estimated_delivery_date",
]

orders_typed = orders_bronze.withColumn("order_status",
F.lower(F.trim(F.col("order_status"))))
for c in order_ts_cols:
    orders_typed = orders_typed.withColumn(c, F.try_to_timestamp(F.col(c),
F.lit(TS_FORMAT)))

# try_to_timestamp hides failures by returning NULL, so count them
for c in order_ts_cols:
    before = orders_bronze.filter(F.col(c).isNotNull()).count()
    after = orders_typed.filter(F.col(c).isNotNull()).count()
    print(f"{c:32} bronze {before:>7,}  typed {after:>7,}  failed {before - after}")
```

8.2 Quality Rules and Quarantine

Each rule writes a reason into `dq_issue`. Rows without a reason go to silver; rows with a reason go to a quarantine table, stamped with `_quarantined_at`.

build_silver (Cell 3)

```
orders_dq = (
    F.when((F.col("order_status") == "delivered") &
F.col("order_delivered_customer_date").isNull(),
           "delivered_no_date")
     .when((F.col("order_status") == "canceled") &
F.col("order_delivered_customer_date").isNotNull(),
           "canceled_with_delivery")
     .otherwise(None)          # NULL label = the row is clean
)

orders_checked = orders_typed.withColumn("dq_issue", orders_dq)
orders_good = orders_checked.filter(F.col("dq_issue").isNull()).drop("dq_issue")
orders_bad = (orders_checked.filter(F.col("dq_issue").isNotNull())
              .withColumn("_quarantined_at", F.current_timestamp()))

orders_good.write.mode("overwrite").option("overwriteSchema",
True).saveAsTable("workspace.silver.orders")
orders_bad.write.mode("overwrite").option("overwriteSchema",
True).saveAsTable("workspace.silver.orders_quarantine")
```

Payments follow the same pattern, with money typed as `DECIMAL(10,2)` so totals are exact to the cent:

03_silver_more (Cell 1, excerpt)

```
pay = (pay_bronze
    .withColumn("payment_value", F.expr("try_cast(payment_value AS DECIMAL(10,2))"))
    .withColumn("payment_type", F.lower(F.trim(F.col("payment_type")))))

pay_dq = (F.when(F.col("payment_value").isNull(), "value_missing_or_invalid")
           .when(F.col("payment_value") < 0, "value_negative")
           .when(F.col("payment_type") == "not_defined", "type_not_defined")
           .otherwise(None))
```

8.3 Deduplication with a Window Function

Some orders had more than one review. `ROW_NUMBER()` over a window keeps only the newest review per order, removing 551 duplicates.

build_silver (Cell 8)

```
w = Window.partitionBy("order_id").orderBy(F.col("review_answer_timestamp").desc())

reviews_silver = (reviews_typed
    .withColumn("rn", F.row_number().over(w))   # 1, 2, 3 ... inside each order
    .filter(F.col("rn") == 1)                    # keep only the newest review
    .drop("rn"))
```

8.4 Flags for Dimensions, Aggregation for Geolocation

Products are a dimension: removing one would orphan every sale that points to it. So suspect weights are **flagged**, not quarantined. Geolocation points outside Brazil's bounding box are quarantined, and

the valid points are averaged to one row per postcode prefix to prevent fan-out in joins.

03_silver_more (Cells 3, 5, 6 excerpt)

```
products = products.withColumn(
    "is_weight_suspect",
    F.col("product_weight_g").isNull() | (F.col("product_weight_g") <= 0)
)

inside_brazil = (F.col("geolocation_lat").between(-34, 6) &
F.col("geolocation_lng").between(-74, -34))

geo_silver = (geo_valid
    .groupBy("geolocation_zip_code_prefix")
    .agg(F.round(F.avg("geolocation_lat"), 6).alias("lat"),
         F.round(F.avg("geolocation_lng"), 6).alias("lng"),
         F.mode("geolocation_city").alias("city"),
         F.mode("geolocation_state").alias("state"),
         F.count("*").alias("points_averaged")))
```

A hidden data issue: the category translation CSV starts with an invisible byte-order mark, which corrupted the first column name. Columns were therefore renamed by position rather than by name.

8.5 Silver Results

Table	Result
orders	99,427 clean + 14 quarantined
payments	103,883 clean + 3 quarantined
reviews	98,673 kept, 551 duplicates removed
products	32,951 (none removed; suspect weights flagged)
sellers	3,095
geolocation	1,000,121 valid + 42 quarantined, aggregated to 19,010 prefixes
customers	99,441

9. GDPR: Personal Data Tokenisation

Olist contains no personal data, so names, emails and phone numbers were simulated with the Faker library, seeded so every run produces the same values. The design keeps personal data in one locked schema and lets only random tokens flow into analytics.

9.1 Synthetic Personal Data

04_customer_pii (Cell 4, excerpt)

```
rows = []
for i, cid in enumerate(ids):
    name = fake.name()
    email = f"{fake.user_name()}{i}@{fake.free_email_domain()}"   # counter keeps emails
unique
    phone = fake.phone_number()
    rows.append((cid, name, email, phone))

profiles_raw = (spark.createDataFrame(rows, "customer_unique_id STRING, full_name STRING,
email STRING, phone STRING")
                .withColumn("_ingested_at", F.current_timestamp())
                .withColumn("_source", F.lit("simulator")))

# Saved in the RESTRICTED schema because it contains personal data
profiles_raw.write.mode("overwrite").saveAsTable("workspace.restricted.customer_profiles_
raw")
```

The same notebook simulates 6,010 address moves into `bronze.customer_address_changes`, which feed the SCD Type 2 customer dimension in gold.

9.2 The Token Vault (Idempotent)

Each personal value receives a random UUID token. A `left_anti` join finds only values that do not yet have a token, so a rerun creates zero new tokens. The vault is append-only.

04_customer_pii (Cell 6)

```
vault = spark.table("workspace.restricted.token_vault")

# LEFT ANTI JOIN = keep only values NOT already in the vault
new_values = values.join(vault, on=["pii_type", "pii_value"], how="left_anti")

# Lazy evaluation trap: count BEFORE writing, or the count is recalculated as 0
n_new = new_values.count()

new_tokens = (new_values
              .withColumn("token", F.expr("uuid()"))     # random ID, no link to the real
value
              .withColumn("created_at", F.current_timestamp()))

new_tokens.write.mode("append").saveAsTable("workspace.restricted.token_vault")   # never
overwrite
```

9.3 Tokenised Silver Profiles

04_customer_pii (Cell 7, excerpt)

```
def tokenise(df, column, pii_type):
    """Swap a personal column for its token column, using the vault."""
    mapping = (vault.filter(F.col("pii_type") == pii_type)
                    .select(F.col("pii_value").alias(column), F.col("token").alias(f"
{column}_token")))
    return df.join(mapping, on=column, how="left").drop(column)

silver_profiles = spark.table("workspace.restricted.customer_profiles_raw")
for column, pii_type in [("full_name", "name"), ("email", "email"), ("phone", "phone")]:
    silver_profiles = tokenise(silver_profiles, column, pii_type)
```

9.4 Access Design and Right to Erasure

- **Least privilege:** analysts receive `SELECT` on silver and gold, and no access to `restricted`. Only a data protection role can read the vault. Free Edition has a single user, so these grants are documented rather than enforced.
- **Right to erasure:** deleting a person's rows from `customer_profiles_raw` and their values from `token_vault` leaves tokens in silver and gold that point to nothing. The person can no longer be identified, and no other table has to be rebuilt.

Access design (documented)

```
-- Analysts can read silver and gold, but never the restricted schema
GRANT USE SCHEMA, SELECT ON SCHEMA workspace.silver TO `analysts`;
REVOKE ALL PRIVILEGES ON SCHEMA workspace.restricted FROM `analysts`;

-- Only the data protection team can read the vault
GRANT USE SCHEMA, SELECT ON SCHEMA workspace.restricted TO `data_protection`;
```

10. dbt Project Setup and Structure

10.1 Local dbt Environment

- Created a Python virtual environment (`.venv`) inside the repository
- Installed `dbt-databricks` (dbt-core 1.12.5, adapter 1.12.8)
- Configured `profiles.yml` **outside** the repository, in the user's home folder
- Read the access token from an environment variable, so it is never written to a file

~/.dbt/profiles.yml (placeholders shown)

```yaml
azure_lakehouse_sales:
  target: dev
  outputs:
    dev:
      type: databricks
      catalog: workspace
      schema: gold
      host: <workspace-host>
      http_path: /sql/1.0/warehouses/<warehouse-id>
      token: "{{ env_var('DATABRICKS_TOKEN') }}"   # read from the terminal, never stored
      threads: 4
```

By default dbt adds the target schema as a prefix (for example `gold_staging`). A small macro override makes models land in clean schema names instead:

dbt/macros/generate_schema_name.sql

```sql
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}
{%- endmacro %}
```

10.2 Repository Structure

```
azure-lakehouse-sales/
├── .github/workflows/ci.yml   → GitHub Actions CI
├── ci/profiles.yml            → dummy dbt profile used only by CI
├── dashboards/                → dashboard SQL queries
├── data/                      → local data folder (raw files not committed)
├── dbt/
│   ├── models/staging/        → 10 stg_* views + sources
│   ├── models/marts/          → star schema (facts and dimensions)
│   ├── models/audit/          → reconciliation report
│   ├── macros/                → generate_schema_name
│   └── tests/                 → singular data tests
├── docs/                      → data model, learnings, images
├── jobs/                      → Databricks Job definition (YAML)
├── notebooks/ and src/        → bronze, silver and PII notebooks
├── terraform/databricks/      → Unity Catalog schemas and volume
├── terraform/azure/           → ADLS Gen2 storage (validated only)
├── requirements-ci.txt, ruff.toml
└── README.md
```

Architectural benefit: each folder has one job, which keeps ingestion, modelling, testing, orchestration and infrastructure clearly separated and easy to review.

11. Gold Layer: dbt Star Schema

dbt is the transformation engine for staging and gold. Silver tables are declared as **sources**, `stg_*` views rename and lightly clean them, and gold models are built with `ref()`, so dbt knows the build order and draws the lineage automatically.

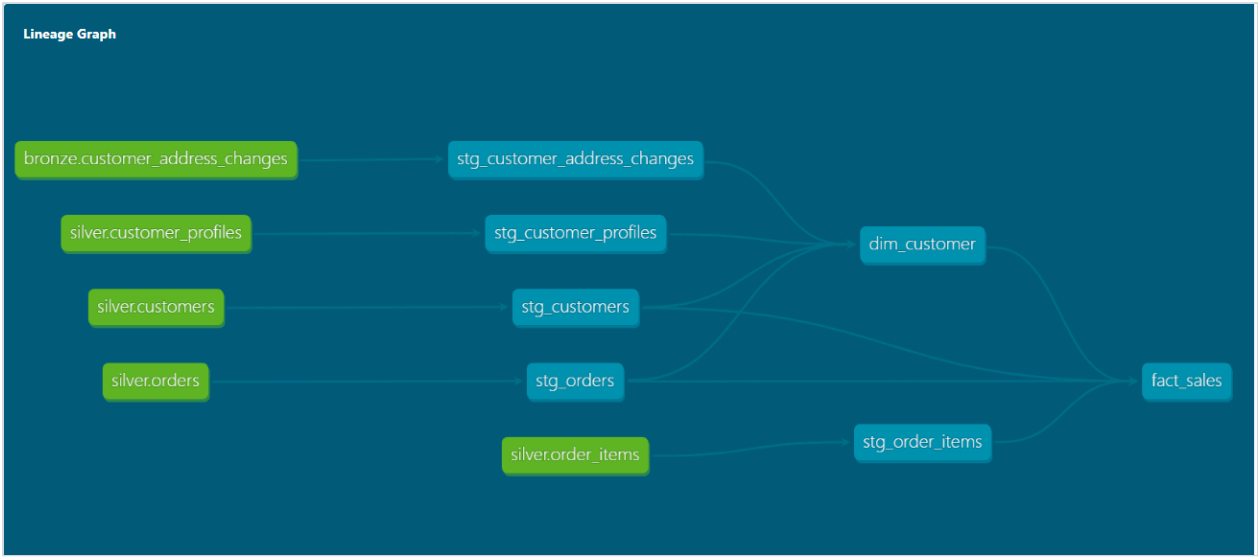


Figure 2. dbt lineage graph for fact_sales: silver sources to staging views to gold

11.1 Star Schema Design

Model	Grain	Key design point
fact_sales	One row per order item (112,635)	Incremental MERGE on sales_key with a 3-day lookback
fct_orders	One row per order (99,427)	Payments aggregated per order before joining
dim_customer	One row per customer version	SCD Type 2 built with LEAD(); as-of join from facts
dim_product	One row per product	English category names; suspect weights flagged
dim_seller	One row per seller	Cleaned location
dim_date	One row per day	Generated date spine

11.2 Incremental Fact Table

The fact table is built incrementally: after the first full build, each run processes only recent orders and merges them into the existing Delta table. Merging on `sales_key` means a re-checked row is updated, never duplicated, so a 3-day lookback can safely catch late-arriving data.

dbt/models/marts/fact_sales.sql

```
{{ config(
    materialized='incremental',
    incremental_strategy='merge',
    unique_key='sales_key'
) }}

select ...                          -- item-level sales with surrogate keys
from {{ ref('stg_order_items') }} as items
...

{% if is_incremental() %}
where purchased_at >= (
    select max(purchased_at) - interval 3 days
    from {{ this }}
)
{% endif %}
```

Simplified excerpt. The full model is in the repository.

Proof that it works: after a full refresh (112,635 rows), the last month was deleted directly in Databricks (6,566 rows, leaving 106,069). A normal `dbt build` without `--full-refresh` merged exactly those rows back to 112,635, and every test still passed.

11.3 Avoiding Fan-out in fct_orders

An order can have several payments. Joining payments directly to items repeats each item once per payment and would have reported 14,209,115.34 BRL for delivered orders instead of the correct 13,220,248.93, about 1 million too high. Payments are therefore aggregated to one row per order first.

dbt/models/marts/fct_orders.sql

```
with payments as (
    select
        order_id,
        sum(payment_value) as total_payment,
        count(*)           as payment_count
    from {{ ref('stg_payments') }}
    group by order_id                -- one row per order BEFORE the join
)

select o.order_id, ..., p.total_payment
from {{ ref('stg_orders') }} as o
left join payments as p
    on o.order_id = p.order_id
```

Simplified excerpt. The full model is in the repository.

11.4 SCD Type 2 Customer Dimension

When a simulated customer moves city, `dim_customer` keeps both versions with valid-from and valid-to dates. `LEAD()` finds when the next version starts, and each sale joins to the version that was current when it was bought.

dbt/models/marts/dim_customer.sql

```
select
    customer_unique_id,
    city,
    state,
    effective_at as valid_from,
    lead(effective_at) over (
        partition by customer_unique_id
        order by effective_at
    ) as valid_to,                   -- NULL = current version
    ...
from address_history

-- in fact_sales: as-of join to the version valid at purchase time
join {{ ref('dim_customer') }} as c
    on  s.customer_unique_id = c.customer_unique_id
    and s.purchased_at >= c.valid_from
    and (s.purchased_at < c.valid_to or c.valid_to is null)
```

Simplified excerpt. The full model is in the repository.

This enables time-correct reporting: revenue by state reflects where each customer lived at the time of purchase.

12. Testing and Reconciliation

Every source row is accounted for from bronze to gold, and the build fails if any layer drifts. The latest full build ran 17 models and 44 data tests:

```
dbt build
```

```
Done. PASS=60 WARN=1 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=61
```

12.1 Tests

Test	What it proves
not_null, unique, accepted_values	Keys and statuses are valid across staging and gold
relationships	Every fact row points to an existing dimension row
assert_sales_match_orders	Item-level revenue equals order-level revenue
assert_one_current_row_per_customer	Each customer has exactly one current SCD2 version
assert_no_overlapping_customer_versions	SCD2 date ranges never overlap
assert_reconciliation_all_passed	Every row of the reconciliation report passes
assert_excluded_items_match_quarantine	Exactly the 15 items of quarantined orders are excluded
assert_missing_customers_match_quarantine	Exactly the 14 customers with only quarantined orders are missing

The single warning is intentional: 15 order items point to quarantined orders. The test is set to `severity: warn` and the gap is documented.

dbt/tests/assert_reconciliation_all_passed.sql

```sql
-- A singular test passes when it returns zero rows
select *
from {{ ref('reconciliation_report') }}
where not check_passed
```

Simplified excerpt.

12.2 Reconciliation Report

The audit model `audit.reconciliation_report` traces every table from bronze to gold and explains each difference as quarantined, removed on purpose (deduplication or exclusion) or aggregated by design.

	table_name	bronze_rows	silver_rows	quarantine_rows	removed_on_purpose	gold_rows	gold_table	check_passed
1	customers	99441	99441	0	14	96082	gold.dim_customer (current rows)	true
2	geolocation	1000163	1000121	42	0	19010	silver.geolocation (one row per postcode pref...	true
3	order_items	112650	112650	0	15	112635	gold.fact_sales	true
4	orders	99441	99427	14	0	99427	gold.fct_orders	true
5	payments	103886	103883	3	0	null	gold.fct_orders (aggregated per order)	true
6	products	32951	32951	0	0	32951	gold.dim_product	true
7	reviews	99224	98673	0	551	null	gold.fct_orders (review_score per order)	true
8	sellers	3095	3095	0	0	3095	gold.dim_seller	true

Figure 3. audit.reconciliation_report in the Databricks SQL Editor: every check passes

Customers fall from 99,441 IDs to 96,082 because Olist creates a new customer ID for each order; gold counts real people (`customer_unique_id`).

12.3 Revenue Trail

Step	Revenue (BRL)	Difference and reason
Silver order items	13,591,643.70	
Gold fact_sales	13,589,748.34	−1,895.36: the 15 items of quarantined orders
Delivered orders only	13,220,248.93	−369,499.41: orders not delivered or cancelled
Fan-out (wrong join, avoided)	14,209,115.34	Would overstate delivered revenue by about 1 million

13. Workflow Orchestration with Databricks Jobs

One Databricks Job, `olist_lakehouse_pipeline`, runs the whole pipeline from raw files to tested gold tables on serverless compute. The run read 17.3 million rows and wrote 2.6 million in 4 minutes 41 seconds.

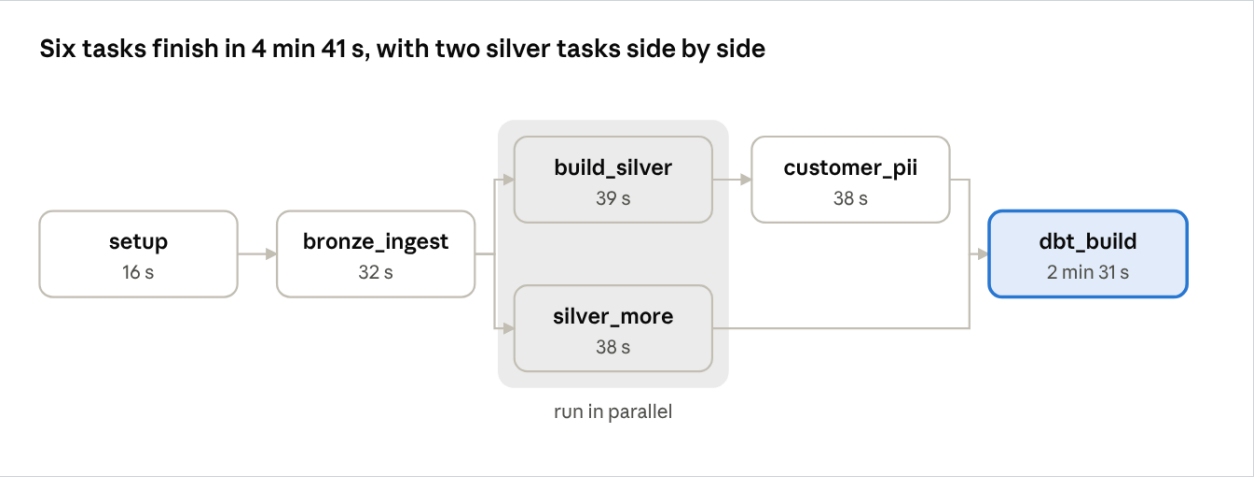


Figure 4. Task graph and run times

Tasks wait only for what they need:

- `silver_more` reads only bronze, so it runs in parallel with `build_silver`
- `customer_pii` needs silver customers, so it waits for `build_silver`
- `dbt_build` waits for both branches, so gold is built only from complete silver data
- If any task fails, the tasks after it do not run, so bad data never reaches gold

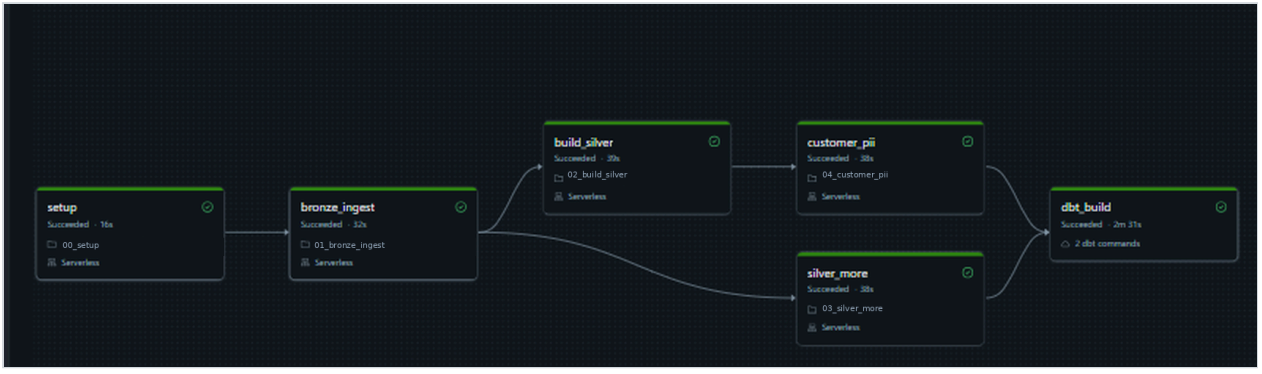


Figure 5. Successful run in Databricks (all 6 tasks green)

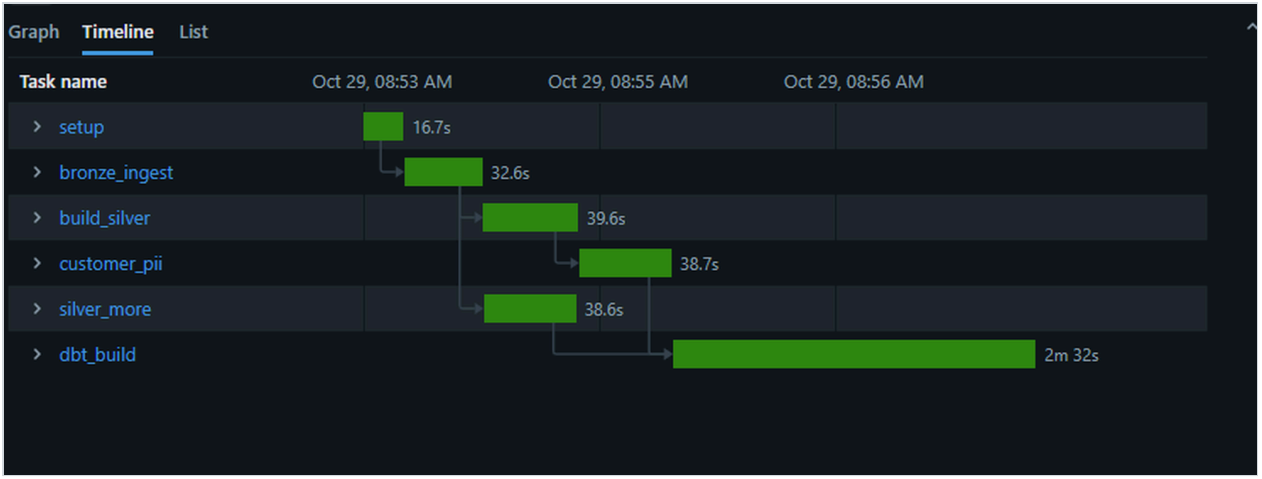


Figure 6. Run timeline: the two silver tasks overlap, then dbt_build runs last

13.1 The dbt Task

- Runs `dbt deps` then `dbt build` straight from the GitHub repository (branch `main`, project directory `dbt`)
- Uses the Serverless Starter Warehouse with catalog `workspace`
- Databricks creates the dbt connection with the job's identity, so no token is stored in the job
- Production always runs the reviewed code on `main`

13.2 Scheduling Decisions

- A daily 06:00 Europe/London schedule is configured and paused to save Free Edition compute
- A suggested table-update trigger was rejected: the job writes the bronze tables itself, so it would restart itself in an endless loop
- The job definition is saved as code in `jobs/olist_lakehouse_pipeline.yml`, with personal details replaced by placeholders

14. CI/CD with GitHub Actions

Every push to `main` and every pull request is checked automatically on a fresh Linux runner. No Databricks credentials are used: dbt is checked against a dummy profile that never connects.

.github/workflows/ci.yml

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:

jobs:
  lint-and-parse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements-ci.txt
      - run: ruff check .
      - run: sqlfluff lint dbt/models dbt/tests
      - run: dbt deps && dbt parse --profiles-dir ../ci
        working-directory: dbt

  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: gitleaks/gitleaks-action@v2

  terraform:          # fmt -check, init -backend=false, validate (both folders)
    ...
```

Simplified excerpt. The full workflow is in the repository.

Check	What it catches
ruff	Python errors and style in notebooks and scripts
sqlfluff (Databricks dialect, Jinja templater)	Inconsistent SQL style in dbt models and tests
dbt deps + dbt parse (dummy profile)	Broken ref(), invalid YAML or missing models
gitleaks	Passwords or tokens committed by mistake
terraform fmt -check + validate	Badly formatted or invalid Terraform

Engineering judgement: a few lint rules were switched off on purpose, each with a comment (for example `RF04`, because `year`, `month` and `state` are real column names). After the SQL was reformatted, a full `dbt build` returned the same PASS=60 result, proving only the layout changed.

One gitleaks finding was an ordinary sentence in the project notes; it was checked and allow-listed by its exact fingerprint rather than by disabling the rule.

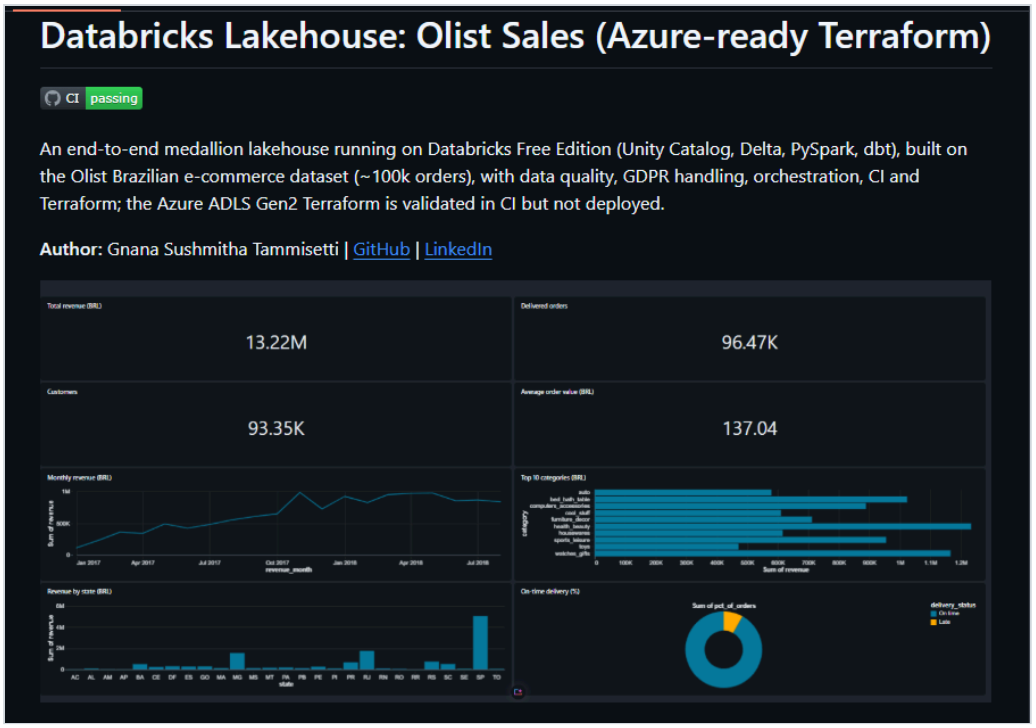


Figure 7. Repository front page with the passing CI badge

15. Infrastructure as Code with Terraform

The Unity Catalog structure that was first created by clicking is now defined in Terraform, so it can be reviewed in Git and recreated with one command.

terraform/databricks/main.tf

```
terraform {
  required_providers {
    databricks = {
      source  = "databricks/databricks"
      version = "~> 1.50"
    }
  }
}

provider "databricks" {}   # reads DATABRICKS_HOST and DATABRICKS_TOKEN from the
environment

import {
  to = databricks_schema.bronze
  id = "workspace.bronze"     # adopt the schema that already exists
}

resource "databricks_schema" "bronze" {
  catalog_name = "workspace"
  name         = "bronze"

  lifecycle {
    prevent_destroy = true    # Terraform can never drop a schema holding data
  }
}
# ... the same for silver, gold, staging, audit, restricted and the landing volume
```

Simplified excerpt. The full configuration is in the repository.

After `terraform apply` imported the 7 objects, a second plan confirmed that the code matches reality:

terraform plan

```
databricks_schema.silver: Refreshing state... [id=workspace.silver]
databricks_schema.gold: Refreshing state... [id=workspace.gold]
...
databricks_volume.landing: Refreshing state... [id=workspace.bronze.landing]

No changes. Your infrastructure matches the configuration.
```

- **No secrets in code:** the provider block is empty and reads credentials from environment variables
- **State protection:** `terraform.tfstate` is git-ignored because it can contain secrets; the provider lock file is committed
- **Azure-ready:** `terraform/azure` defines a resource group, an ADLS Gen2 storage account (hierarchical namespace, TLS 1.2, no public access) and bronze, silver and gold containers. It is validated in CI but not deployed, because the project runs on Databricks Free Edition.

16. Dashboard with Databricks AI/BI

The dashboard **Olist Sales Overview** reads the gold star schema directly. Its headline revenue of 13.22 million BRL matches the dbt reconciliation exactly, so the dashboard and the pipeline agree.

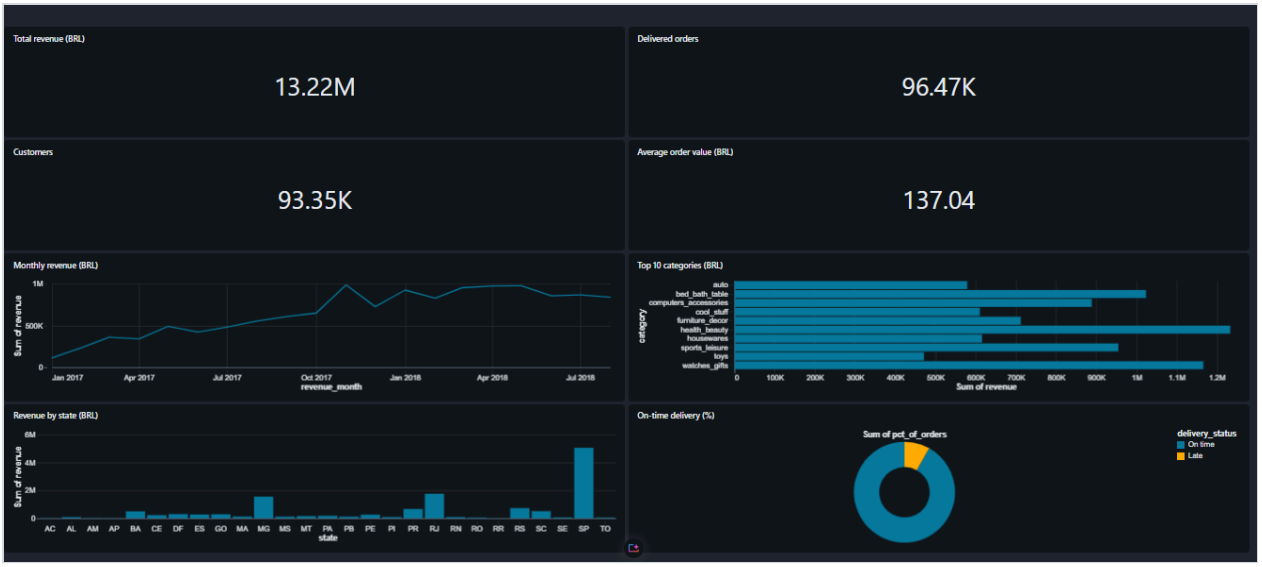


Figure 8. Olist Sales Overview dashboard

16.1 Key Performance Indicators

- Delivered revenue: 13.22M BRL
- Delivered orders: 96.47K
- Customers: 93.35K
- Average order value: 137.04 BRL

dashboards/sales_dashboard.sql (query 1)

```sql
select
    round(sum(price), 2)                          as total_revenue,
    count(distinct order_id)                      as total_orders,
    count(distinct customer_unique_id)            as total_customers,
    round(sum(price) / count(distinct order_id), 2) as average_order_value
from workspace.gold.fact_sales
where is_delivered
```

16.2 Visual Analytics

Visual	Insight
Monthly revenue (line)	Revenue grows through 2017 and levels off near 1M BRL a month in 2018
Top 10 categories (bar)	health_beauty and watches_gifts lead revenue
Revenue by state (bar)	São Paulo (SP) is by far the largest market, at about 5M BRL
On-time delivery (pie)	About 92% of delivered orders arrive by the estimated date

dashboards/sales_dashboard.sql (query 5)

```
select
    case when is_late then 'Late' else 'On time' end as delivery_status,
    count(*) as orders,
    round(100.0 * count(*) / sum(count(*)) over (), 1) as pct_of_orders
from workspace.gold.fct_orders
where order_status = 'delivered'
  and is_late is not null          -- orders with no delivery date are not counted as on
time
group by case when is_late then 'Late' else 'On time' end
order by orders desc
```

The monthly chart is limited to January 2017 to August 2018, because the dataset has very few orders before and after that period. Revenue by state uses the SCD Type 2 join, so each sale counts where the customer lived at the time.

17. Key Learnings and Challenges

17.1 Learnings and Best Practices

- Medallion architecture on Delta Lake and Unity Catalog
- Quarantine instead of silent deletion, with a reconciliation report that proves completeness
- Choosing the right grain to avoid fan-out in joins
- dbt incremental models with MERGE and SCD Type 2 dimensions
- Privacy by design with tokenisation and least-privilege access
- Orchestration with task dependencies, CI/CD and Infrastructure as Code
- Keeping secrets out of code: environment variables, dummy CI profiles and git-ignored state

17.2 Real Challenges and Fixes

Challenge	Fix
Joining payments to items inflated revenue	Aggregated payments to one row per order first
An invisible byte-order mark broke a column name	Renamed the columns by position
Review comments with line breaks split rows	Read CSVs with multiLine and escape options
An incremental test looked like a failure	The count was checked before the build finished; the passing tests confirmed the result
Hidden spaces in a Databricks job field	Retyped the project directory by hand
gitleaks flagged a sentence in the notes	Verified it and allow-listed only that exact fingerprint
A token kept failing in Terraform	Found that copying a command replaced the token on the clipboard; verified secrets by length and first 4 characters only

Known gaps, documented openly: 15 order items and 14 customers are excluded because their only orders were quarantined (both tested to be exactly those rows); personal data and address moves are simulated; access grants are documented rather than enforced on single-user Free Edition; changes older than the 3-day lookback need a full refresh.

18. Conclusion

This project delivers a complete lakehouse on Databricks, integrating:

- Reliable ingestion with full reconciliation from source to gold
- Data quality rules, quarantine and deduplication
- A dbt star schema with SCD Type 2 history and incremental loading
- GDPR-style tokenisation of personal data
- Automated orchestration, CI/CD and Infrastructure as Code
- A business dashboard whose numbers tie back to the pipeline exactly

Above all, every number in the final dashboard can be traced to the source data and explained, which is what makes a data platform trustworthy for the business.

19. Future Enhancements

- Deploy the Azure Terraform (ADLS Gen2) and read bronze from cloud storage
- Stream new orders with Auto Loader or Structured Streaming instead of batch files
- Use Lakeflow Declarative Pipelines expectations for in-pipeline data quality

- Enforce access with Unity Catalog groups, row filters and column masks in a multi-user workspace
- Add alerting on job failures and data freshness checks
- Add separate dev and prod environments with Databricks Asset Bundles